



ΟΙΚΟΝΟΜΙΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

«ΕΡΓΑΣΙΑ ΣΤΟ ΜΑΘΗΜΑ ΤΕΧΝΗΤΗ ΝΟΗΜΟΣΥΝΗ»

Διάσχιση γέφυρας

Γιαννακοπούλου Αριάδνη, 3210028

Μπελεγράτη Μαριάννα, 3210131

Σύρμα Ιωάννα, 3210190

Διδάσκων: Ι. Ανδρουτσόπουλος

Αθήνα

Χειμερινό εξάμηνο 2023

Στόχος αυτής της εργασίας είναι να επιλύσει το πρόβλημα του Bridge Crossing με τη χρήση του A* αλγορίθμου σε γλώσσα προγραμματισμού Java.

Ο χρήστης είναι αυτός που θα καταχωρήσει τον αριθμό των ατόμων που συμμετέχουν καθώς και τον χρόνο που χρειάζεται ο καθένας από αυτούς για να περάσει στην απέναντι όχθη. Τα δεδομένα για να μπορέσουν να επεξεργαστούν, αποθηκεύονται σε ArrayList(right,left,membersTime). Η λίστα left αρχικοποιείται με 0 για όλα τα μέλη και η λίστα right με 1. Αν βρεθεί τελική λύση τότε όλα τα στοιχεία του πίνακα left θα είναι 1 και του right 0 και θα σημαίνει ότι όλα τα μέλη έχουν καταφέρει να περάσουν την γέφυρα μέσα στον μέγιστο χρόνο(T), ο οποίος δηλώνεται από την αρχή ότι είναι 30 λεπτά.

```
Enter number of family members:
5
The family members have 30 minutes to pass the bridge
Enter minutes of traversal of family member No1:
1
Enter minutes of traversal of family member No2:
3
Enter minutes of traversal of family member No3:
6
Enter minutes of traversal of family member No4:
8
Enter minutes of traversal of family member No5:
12
```

Σε αυτά τα 30 λεπτά μπορούν μόνο 2 άτομα να περνάνε τον κορμό που συνδέει τις 2 όχθες κρατώντας την μοναδική λάμπα που έχουν στην κατοχή τους και σε αυτή τη διαδρομή μετράμε τον χρόνο διάσχισης του βραδύτερου. Αργότερα ένας από την αριστερή πλευρά πηγαίνει δεξιά με την λάμπα ώστε να περάσουν πάλι δύο άτομα απέναντι. Ζητείται να βρεθεί το ελάχιστο μονοπάτι. Για να βρεθεί το μονοπάτι αυτό με την βοήθεια του A* αλγορίθμου, χρειάζεται να δημιουργηθούν όλα τα δυνατά στιγμιότυπα από κάθε μία κατάσταση, ξεκινώντας από την αρχική που όλα τα μέλη βρίσκονται στην δεξιά μεριά της γέφυρας. Στην συνέχεια κάθε στιγμιότυπο εξετάζεται από τον A* και προστίθεται στο βέλτιστο μονοπάτι βάση της συνάρτησης $f(u) = g(u) + h(u)$, όπου u το κάθε στιγμιότυπο, g ο χρόνος που έχει περάσει από την αρχική κατάσταση μέχρι την κατάσταση που ελέγχουμε και h ένας αριθμός που υπολογίζεται με την χρήση της heuristic(), η οποία δείχνει πόσα άτομα απομένουν ακόμα για να περάσουν από την δεξιά στην αριστερή μεριά(γενικότερα, η heuristic/ευρετική συνάρτηση, βοηθάει στο να υπολογίσουμε το κόστος μέχρι την τελική κατάσταση από μία προηγούμενη κατάσταση που ελέγχουμε).

Για την υλοποίηση χρησιμοποιούμε δύο βασικές κλάσεις. Η πρώτη κλάση State δημιουργεί τις καταστάσεις του παιχνιδιού αποθηκεύοντας τα f, g, h, τον “πατέρα”για καθεμία, καθώς και άλλες απαραίτητες πληροφορίες. Μέσα σε αυτή την κλάση υπάρχουν μέθοδοι όπως goLeft και goRight, που χρησιμοποιούνται όταν μετακινούνται τα μέλη αριστερά και δεξιά αντίστοιχα, κάνοντας τις απαραίτητες αλλαγές. Επίσης, περιλαμβάνει την μέθοδο getChildren η οποία δημιουργεί όλους τους δυνατούς συνδυασμούς καταστάσεων(“παιδιά” της κατάστασης που δίνεται- επόμενες εφικτές κινήσεις), ώστε να επιλεγεί η πιο συμφέρουσα. Ακόμη, γίνεται χρήση της μεθόδου heuristic (ευρετική), η οποία υπολογίζει τον αριθμό h, με την βοήθεια της μεθόδου peopleSide, όπως και της evaluate, η οποία υπολογίζει τον αριθμό f. Ιδιαίτερα σημαντική είναι και η compareTo, η οποία χρησιμοποιείται για την σύγκριση των f δύο καταστάσεων.

Η δεύτερη βασική κλάση είναι η AStar, που υλοποιεί τον αλγόριθμο. Σε αυτή διατηρούνται ένα ArrayList (frontier), στο οποίο αποθηκεύονται τα παιδιά κάθε κατάστασης που πρέπει να εξεταστούν, και ένα HashSet (closedSet), στο οποίο προστίθεται σε κάθε επανάληψη η κατάσταση(currentState)

που εξετάζονται τα παιδιά της, ελέγχοντας αν έχει ξανά εξεταστεί η ίδια. Το ArrayList frontier ταξινομείται σε κάθε επανάληψη με βάση την τιμή της f, ώστε να επιλέγεται στην επόμενη η κατάσταση που μας συμφέρει περισσότερο(αυτή με το μικρότερο κόστος -f). Ταυτόχρονα στον αλγόριθμο της AStar ελέγχουμε αν έχουμε ξεπεράσει το totalTime, που έχουμε θέσει στην Main, για να τερματίσουμε το παιχνίδι χωρίς λύση καθώς και αν βρισκόμαστε στην τελική κατάσταση-στόχο(όλα τα μέλη αριστερά), ώστε να τερματίσουμε εμφανίζοντας την διαδρομή-λύση που ακολούθησε ο αλγόριθμος, μαζί με τον χρόνο που χρειάστηκαν τα μέλη.

Για την εκτύπωση του βέλτιστου μονοπατιού, χρησιμοποιήθηκε η μέθοδος backtracking απο την τελική κατάσταση, αποθηκεύοντας σε ένα ArrayList το μονοπάτι που ακολουθήθηκε, δηλαδή για κάθε “παιδί”τον “πατέρα” του και στην συνέχεια τον “πατέρα” του “πατέρα” κλπ. Έπειτα εκτυπώνουμε το αντίστροφο του.

```
The game started!
~
LEFT SIDE

RIGHT SIDE
Member1
Member2
Member3
Member4
Member5

The lamp is right!
Time elapsed: 0 minutes
~
~
LEFT SIDE
Member1
Member2

RIGHT SIDE
Member3
Member4
Member5

The lamp is left!
Time elapsed: 3 minutes
~
~
LEFT SIDE
Member2

RIGHT SIDE
Member1
Member3
Member4
Member5

The lamp is right!
Time elapsed: 4 minutes
```

```
~~~~~  
LEFT SIDE  
Member2  
Member4  
Member5
```

```
RIGHT SIDE  
Member1  
Member3
```

```
The lamp is left!  
Time elapsed: 16 minutes
```

```
~~~~~  
LEFT SIDE  
Member4  
Member5
```

```
RIGHT SIDE  
Member1  
Member2  
Member3
```

```
The lamp is right!  
Time elapsed: 19 minutes
```

```
~~~~~
```

~~~~~

LEFT SIDE

Member1

Member2

Member4

Member5

RIGHT SIDE

Member3

The lamp is left!

Time elapsed: 22 minutes

~~~~~

LEFT SIDE

Member2

Member4

Member5

RIGHT SIDE

Member1

Member3

The lamp is right!

Time elapsed: 23 minutes

~~~~~

FINAL STATE

~~~~~

LEFT SIDE

Member1

Member2

Member3

Member4

Member5

RIGHT SIDE

~~~~~

All the members have passed the bridge!

They needed 29 minutes!